WEEK-8

Title

Online Banking System using Node.js and Express.js

Aim

To develop a web-based Online Banking System that allows users to securely register, login, manage their profile, check account balance, perform banking transactions, and generate transaction receipts.

Procedure

1. Create a Node.js project and initialize it using npm init.
2. Install the required dependencies such as Express.js.
3. Design the banking system using HTML and CSS web pages.
4. Create registration and login modules for user authentication.
5. Implement the backend using Node.js and Express.js.
6. Store user details and transaction information using JSON files.
7. Develop the dashboard to display account and transaction details.
8. Implement banking operations such as deposit, withdrawal, and transfer.
9. Add profile, change-password, and transaction-receipt functionalities.
10. Start the server and test all modules through the web browser.

App.js

```
const express = require("express");
const session = require("express-session");
const bcrypt = require("bcryptjs");
const helmet = require("helmet");
const rateLimit = require("express-rate-limit");
const fs = require("fs");
const path = require("path");
require("dotenv").config();

const app = express();
const PORT = process.env.PORT || 3000;

const USERS_FILE = path.join(__dirname, "users.json");
const TRANSACTIONS_FILE = path.join(__dirname, "transactions.json");

app.use(helmet());
app.use(express.json());
app.use(express.urlencoded({ extended: false }));

const loginLimiter = rateLimit({
    windowMs: 15 * 60 * 1000,
    max: 10,
    message: {
        success: false,
        message: "Too many login attempts. Please try again later."
    }
});

app.use(session({
    secret: process.env.SESSION_SECRET || "change-this-secret",
    resave: false,
    saveUninitialized: false,
    cookie: {
        httpOnly: true,
        sameSite: "lax",
        secure: process.env.NODE_ENV === "production",
        maxAge: 30 * 60 * 1000
    }
}));
```

```
app.use(express.static(path.join(__dirname, "public")));

function readJSON(file) {
  try {
    return JSON.parse(fs.readFileSync(file, "utf8"));
  } catch {
    return [];
  }
}

function writeJSON(file, data) {
  fs.writeFileSync(file, JSON.stringify(data, null, 2));
}

function generateAccountNumber(users) {
  let accountNumber;
  do {
    accountNumber =
      "MGH" + Math.floor(1000000000 + Math.random() * 9000000000);
  } while (users.some(u => u.accountNumber === accountNumber));
  return accountNumber;
}

function generateTransactionId() {
  return "TXN" + Date.now() + Math.floor(Math.random() * 1000);
}

function requireLogin(req, res, next) {
  if (!req.session.userId) {
    return res.status(401).json({
      success: false,
      message: "Please login first."
    });
  }
  next();
}

app.get("/", (req, res) => {
  res.sendFile(path.join(__dirname, "public", "index.html"));
```

```
});

/* REGISTER */
app.post("/api/register", async (req, res) => {
  try {
    const {
      name, dob, gender, email, mobile,
      username, password, address
    } = req.body;

    if (!name || !dob || !gender || !email || !mobile ||
      !username || !password || !address) {
      return res.status(400).json({
        success: false,
        message: "All fields are required."
      });
    }

    if (password.length < 8) {
      return res.status(400).json({
        success: false,
        message: "Password must contain at least 8 characters."
      });
    }

    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailPattern.test(email)) {
      return res.status(400).json({
        success: false,
        message: "Enter a valid email address."
      });
    }

    const mobilePattern = /^[6-9]\d{9}$/;
    if (!mobilePattern.test(mobile)) {
      return res.status(400).json({
        success: false,
        message: "Enter a valid 10-digit mobile number."
      });
    }
```

```
    const users = readJSON(USERS_FILE);
    const existingUser = users.find(user =>
      user.username.toLowerCase() === username.toLowerCase() ||
      user.email.toLowerCase() === email.toLowerCase()
    );

    if (existingUser) {
      return res.status(409).json({
        success: false,
        message: "Username or email already registered."
      });
    }

    const passwordHash = await bcrypt.hash(password, 12);

    const newUser = {
      id: Date.now().toString(),
      name,
      dob,
      gender,
      email,
      mobile,
      username,
      passwordHash,
      address,
      accountNumber: generateAccountNumber(users),
      balance: 10000,
      createdAt: new Date().toISOString()
    };

    users.push(newUser);
    writeJSON(USERS_FILE, users);

    res.status(201).json({
      success: true,
      message: "Meghana Bank account created successfully.",
      accountNumber: newUser.accountNumber
    });
  } catch (error) {
```

```
        console.error(error);
        res.status(500).json({ success: false, message: "Server error." });
    }
});

/* LOGIN */
app.post("/api/login", loginLimiter, async (req, res) => {
    try {
        const { username, password } = req.body;

        if (!username || !password) {
            return res.status(400).json({
                success: false,
                message: "Username and password are required."
            });
        }

        const users = readJSON(USERS_FILE);
        const user = users.find(u =>
            u.username.toLowerCase() === username.toLowerCase() ||
            u.email.toLowerCase() === username.toLowerCase()
        );

        if (!user) {
            return res.status(401).json({
                success: false,
                message: "Invalid username/email or password."
            });
        }

        const passwordMatch = await bcrypt.compare(password, user.passwordHash);

        if (!passwordMatch) {
            return res.status(401).json({
                success: false,
                message: "Invalid username/email or password."
            });
        }

        req.session.userId = user.id;
```

```
        res.json({ success: true, message: "Login successful." });
    } catch (error) {
        console.error(error);
        res.status(500).json({ success: false, message: "Server error." });
    }
});

/* CURRENT USER */
app.get("/api/me", requireLogin, (req, res) => {
    const users = readJSON(USERS_FILE);
    const user = users.find(u => u.id === req.session.userId);

    if (!user) {
        return res.status(404).json({
            success: false,
            message: "User not found."
        });
    }

    res.json({
        success: true,
        user: {
            name: user.name,
            email: user.email,
            mobile: user.mobile,
            username: user.username,
            accountNumber: user.accountNumber,
            balance: user.balance,
            dob: user.dob,
            gender: user.gender,
            address: user.address
        }
    });
});

/* LOGOUT */
app.post("/api/logout", (req, res) => {
    req.session.destroy(() => {
        res.clearCookie("connect.sid");
```

```
        res.json({
            success: true,
            message: "Logged out successfully."
        });
    });
});

/* TRANSACTIONS */
app.get("/api/transactions", requireLogin, (req, res) => {
    const transactions = readJSON(TRANSACTIONS_FILE);
    const userTransactions = transactions.filter(
        txn => txn.userId === req.session.userId
    );

    res.json({ success: true, transactions: userTransactions });
});

/* CHANGE PASSWORD */
app.post("/api/change-password", requireLogin, async (req, res) => {
    try {
        const { currentPassword, newPassword } = req.body;

        if (!currentPassword || !newPassword) {
            return res.status(400).json({
                success: false,
                message: "All password fields are required."
            });
        }

        if (newPassword.length < 8) {
            return res.status(400).json({
                success: false,
                message: "New password must contain at least 8 characters."
            });
        }

        const users = readJSON(USERS_FILE);
        const user = users.find(u => u.id === req.session.userId);

        if (!user) {
```

```
            return res.status(404).json({
                success: false,
                message: "User account not found."
            });
        }

        const valid = await bcrypt.compare(currentPassword, user.passwordHash);

        if (!valid) {
            return res.status(401).json({
                success: false,
                message: "Current password is incorrect."
            });
        }

        user.passwordHash = await bcrypt.hash(newPassword, 12);
        writeJSON(USERS_FILE, users);

        res.json({
            success: true,
            message: "Password changed successfully."
        });
    } catch (error) {
        console.error(error);
        res.status(500).json({ success: false, message: "Server error." });
    }
});

/* DEPOSIT */
app.post("/api/deposit", requireLogin, (req, res) => {
    const amount = Number(req.body.amount);

    if (!Number.isFinite(amount) || amount <= 0) {
        return res.status(400).json({
            success: false,
            message: "Invalid deposit amount."
        });
    }

    const users = readJSON(USERS_FILE);
```

```
    const transactions = readJSON(TRANSACTIONS_FILE);
    const user = users.find(u => u.id === req.session.userId);

    if (!user) {
        return res.status(404).json({
            success: false,
            message: "Account not found."
        });
    }

    user.balance += amount;

    const transaction = {
        transactionId: generateTransactionId(),
        userId: user.id,
        type: "DEPOSIT",
        amount,
        date: new Date().toISOString()
    };

    transactions.push(transaction);
    writeJSON(USERS_FILE, users);
    writeJSON(TRANSACTIONS_FILE, transactions);

    res.json({
        success: true,
        message: `₹${amount.toFixed(2)} deposited successfully.`,
        transaction
    });
});

/* WITHDRAW */
app.post("/api/withdraw", requireLogin, (req, res) => {
    const amount = Number(req.body.amount);

    if (!Number.isFinite(amount) || amount <= 0) {
        return res.status(400).json({
            success: false,
            message: "Invalid withdrawal amount."
        });
```

```
    }

    const users = readJSON(USERS_FILE);
    const transactions = readJSON(TRANSACTIONS_FILE);
    const user = users.find(u => u.id === req.session.userId);

    if (!user) {
        return res.status(404).json({
            success: false,
            message: "Account not found."
        });
    }

    if (amount > user.balance) {
        return res.status(400).json({
            success: false,
            message: "Insufficient account balance."
        });
    }

    user.balance -= amount;

    const transaction = {
        transactionId: generateTransactionId(),
        userId: user.id,
        type: "WITHDRAW",
        amount,
        date: new Date().toISOString()
    };

    transactions.push(transaction);
    writeJSON(USERS_FILE, users);
    writeJSON(TRANSACTIONS_FILE, transactions);

    res.json({
        success: true,
        message: `₹${amount.toFixed(2)} withdrawn successfully.`,
        transaction
    });
});
```

```
/* TRANSFER */
app.post("/api/transfer", requireLogin, (req, res) => {
    const amount = Number(req.body.amount);
    const receiverAccount = String(req.body.receiverAccount || "").trim();

    if (!Number.isFinite(amount) || amount <= 0) {
        return res.status(400).json({
            success: false,
            message: "Invalid transfer amount."
        });
    }

    if (!receiverAccount) {
        return res.status(400).json({
            success: false,
            message: "Receiver account is required."
        });
    }

    const users = readJSON(USERS_FILE);
    const transactions = readJSON(TRANSACTIONS_FILE);

    const sender = users.find(u => u.id === req.session.userId);

    if (!sender) {
        return res.status(404).json({
            success: false,
            message: "Sender account not found."
        });
    }

    const receiver = users.find(u => u.accountNumber === receiverAccount);

    if (!receiver) {
        return res.status(404).json({
            success: false,
            message: "Receiver account not found."
        });
    }
```

```
if (sender.accountNumber === receiver.accountNumber) {
    return res.status(400).json({
        success: false,
        message: "You cannot transfer money to yourself."
    });
}

if (amount > sender.balance) {
    return res.status(400).json({
        success: false,
        message: "Insufficient account balance."
    });
}

sender.balance -= amount;
receiver.balance += amount;

const transactionId = generateTransactionId();

const senderTransaction = {
    transactionId,
    userId: sender.id,
    type: "TRANSFER",
    amount,
    receiverAccount: receiver.accountNumber,
    date: new Date().toISOString()
};

const receiverTransaction = {
    transactionId: transactionId + "-R",
    userId: receiver.id,
    type: "RECEIVED",
    amount,
    senderAccount: sender.accountNumber,
    date: new Date().toISOString()
};

transactions.push(senderTransaction, receiverTransaction);
```

```
    writeJSON(USERS_FILE, users);
    writeJSON(TRANSACTIONS_FILE, transactions);

    res.json({
      success: true,
      message: `₹${amount.toFixed(2)} transferred successfully.`,
      transaction: senderTransaction
    });
});

/* SERVER - KEEP THIS AT THE VERY END */
app.listen(PORT, "0.0.0.0", () => {
    console.log(`Meghana Bank running at http://localhost:${PORT}`);
});
```

Package.json

```
{
  "name": "meghana-bank",
  "version": "1.0.0",
  "description": "Meghana Bank Online Banking Demo",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "bcryptjs": "^3.0.2",
    "dotenv": "^17.2.2",
    "express": "^5.2.1",
    "express-rate-limit": "^8.7.0",
    "express-session": "^1.18.2",
    "helmet": "^8.1.0"
  }
}
```

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Meghana Bank | Online Banking</title>
```

```
<link rel="stylesheet" href="css/style.css">
</head>
<body>
<nav class="navbar">
  <div class="brand">MEGHANA <span>BANK</span></div>
  <div class="nav-links">
    <a href="index.html">Home</a>
    <a href="register.html">Register</a>
    <a href="login.html">Login</a>
  </div>
</nav>

<section class="hero">
  <div class="hero-text">
    <p class="eyebrow">SMART • SIMPLE • SECURE</p>
    <h1>WELCOME TO <span>MEGHANA BANK</span></h1>
    <p>Modern online banking for secure account management, deposits, withdrawals and transfers.</p>
    <div class="hero-buttons">
      <a class="btn primary" href="register.html">Open Account</a>
      <a class="btn secondary" href="login.html">Login</a>
    </div>
  </div>
  <div class="bank-card">
    <div class="card-top"><span>MEGHANA BANK</span><span>VISA</span></div>
    <div class="chip"></div>
    <div class="card-number">**** **** **** 2041</div>
    <div class="card-bottom"><span>MEGHANA CUSTOMER</span><span>ACTIVE</span></div>
  </div>
</section>

<section class="features">
  <div class="feature"><h3>Secure Login</h3><p>Password hashing and session-based authentication.</p></div>
  <div class="feature"><h3>Easy Transfers</h3><p>Transfer funds using a Meghana Bank account number.</p></div>
  <div class="feature"><h3>Transactions</h3><p>View your banking transaction history anytime.</p></div>
</section>

<footer>© 2026 MEGHANA BANK • Online Banking Demo Project</footer>
</body>
</html>
```

Dashboard.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Meghana Bank | Dashboard</title>
<link rel="stylesheet" href="css/style.css">
</head>
<body>
<nav class="navbar">
  <div class="brand">MEGHANA <span>BANK</span></div>
  <div class="nav-links">
    <a href="dashboard.html">Dashboard</a>
    <a href="profile.html">Profile</a>
    <a href="change-password.html">Change Password</a>
    <button id="logoutBtn" class="logout-btn">Logout</button>
  </div>
</nav>

<main class="dashboard">
  <div class="dashboard-heading">
    <div><p class="eyebrow">PERSONAL BANKING</p><h1>Welcome, <span id="userName">Customer</span></h1></div>
    <span class="secure-badge">✓ Secure Session</span>
  </div>

  <section class="account-card">
    <div>
      <p>AVAILABLE BALANCE</p>
      <h2 id="balance">₹0.00</h2>
      <small>Account: <span id="accountNumber">-</span></small>
    </div>
    <div class="account-mark">MB</div>
  </section>

  <section class="stats-grid">
    <div class="stat-card"><span>Total Deposits</span><strong id="totalDeposits">₹0.00</strong></div>
    <div class="stat-card"><span>Total Withdrawals</span><strong id="totalWithdrawals">₹0.00</strong></div>
    <div class="stat-card"><span>Total Transfers</span><strong id="totalTransfers">₹0.00</strong></div>
    <div class="stat-card"><span>Transactions</span><strong id="transactionCount">0</strong></div>
  </section>
```

```
 <section class="actions">
  <div class="action-card">
   <h3>Deposit Money</h3>
   <p>Add money to your account.</p>
   <button id="depositBtn" class="btn primary">Deposit</button>
  </div>
  <div class="action-card">
   <h3>Withdraw Money</h3>
   <p>Withdraw money from your account.</p>
   <button id="withdrawBtn" class="btn primary">Withdraw</button>
  </div>
  <div class="action-card">
   <h3>Transfer Money</h3>
   <p>Send money to another Meghana account.</p>
   <button id="transferBtn" class="btn primary">Transfer</button>
  </div>
 </section>

 <section class="transaction-section">
  <div class="section-head"><h2>Recent Transactions</h2><button id="refreshBtn" class="btn secondary">Refresh</button></div>
  <div class="table-wrap">
   <table>
    <thead><tr><th>ID</th><th>Type</th><th>Amount</th><th>Date</th></tr></thead>
    <tbody id="transactionBody"><tr><td colspan="4">Loading...</td></tr></tbody>
   </table>
  </div>
 </section>
</main>
<script src="js/dashboard.js"></script>
</body>
</html>
```

Change-password.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Meghana Bank | Change Password</title>
```

<link rel="stylesheet" href="css/style.css">

</head>

<body class="form-page">

<nav class="navbar">

  MEGHANA BANK

  DashboardProfile<button id="logoutBtn" class="logout-btn">Logout</button>

</nav>

  <h1>Change Password</h1>

  Update your Meghana Bank login password.

  <form id="passwordForm">

    <label>Current Password<input type="password" name="currentPassword" required></label>

    <label>New Password<input type="password" name="newPassword" minlength="8" required></label>

    <label>Confirm New Password<input type="password" name="confirmPassword" minlength="8" required></label>

    <button class="btn primary full" type="submit">Change Password</button>

  </form>

<script src="js/change-password.js"></script>

</body>

</html>

Login.html

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Meghana Bank | Login</title>

<link rel="stylesheet" href="css/style.css">

</head>

<body class="form-page">

<nav class="navbar">

  MEGHANA BANK

  HomeRegister

</nav>

  🔐

```
  <h1>Welcome Back</h1>
  <p class="muted">Login to your Meghana Bank account</p>
  <div id="message"></div>
  <form id="loginForm">
    <label>Username or Email<input name="username" required autocomplete="username"></label>
    <label>Password<input type="password" name="password" required autocomplete="current-password"></label>
    <button class="btn primary full" type="submit">Login Securely</button>
  </form>
  <p class="center">New customer? <a href="register.html">Create account</a></p>
  <div class="security-note">Your connection is protected with session-based authentication.</div>
</div>
<script src="js/login.js"></script>
</body>
</html>
```

Profile.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Meghana Bank | Profile</title>
<link rel="stylesheet" href="css/style.css">
</head>
<body>
<nav class="navbar">
  <div class="brand">MEGHANA <span>BANK</span></div>
  <div class="nav-links"><a href="dashboard.html">Dashboard</a><a href="change-password.html">Change Password</a><button id="logoutBtn" class="logout-btn">Logout</button></div>
</nav>
<div class="profile-container">
  <h1>My Profile</h1>
  <div id="profile"></div>
</div>
<script src="js/profile.js"></script>
</body>
</html>
```

Register.html

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Meghana Bank | Register</title>
<link rel="stylesheet" href="css/style.css">
</head>
<body class="form-page">
<nav class="navbar">
  <div class="brand">MEGHANA <span>BANK</span></div>
  <div class="nav-links"><a href="index.html">Home</a><a href="login.html">Login</a></div>
</nav>

<div class="form-container">
  <h1>Open Meghana Bank Account</h1>
  <p class="muted">Create your online banking account</p>
  <div id="message"></div>

  <form id="registerForm">
    <div class="grid-2">
      <label>Full Name<input name="name" required></label>
      <label>Date of Birth<input type="date" name="dob" required></label>
      <label>Gender<select name="gender" required><option
value="">Select</option><option>Male</option><option>Female</option><option>Other</option></select></label>
      <label>Email<input type="email" name="email" required></label>
      <label>Mobile Number<input name="mobile" inputmode="numeric" maxlength="10" required></label>
      <label>Username<input name="username" required></label>
      <label>Password<input type="password" name="password" minlength="8" required></label>
      <label>Confirm Password<input type="password" name="confirmPassword" minlength="8" required></label>
    </div>
    <label>Address<textarea name="address" rows="3" required></textarea></label>
    <button class="btn primary full" type="submit">Create Account</button>
  </form>
  <p class="center">Already registered? <a href="login.html">Login here</a></p>
</div>
<script src="js/register.js"></script>
</body>
</html>
```

JS FILES

Change-password.js

```
const form = document.getElementById("passwordForm");
const message = document.getElementById("message");

form.addEventListener("submit", async (e) => {
  e.preventDefault();

  const data = Object.fromEntries(new FormData(form).entries());

  if (data.newPassword !== data.confirmPassword) {
    message.innerHTML = '<div class="error">New passwords do not match.</div>';
    return;
  }

  try {
    const response = await fetch("/api/change-password", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        currentPassword: data.currentPassword,
        newPassword: data.newPassword
      })
    });

    const result = await response.json();

    if (!result.success) {
      message.innerHTML = `<div class="error">${result.message}</div>`;
      return;
    }

    message.innerHTML = `<div class="success">${result.message}</div>`;
    form.reset();
  } catch {
    message.innerHTML = '<div class="error">Unable to connect to server.</div>';
  }
});

document.getElementById("logoutBtn").addEventListener("click", async () => {
  await fetch("/api/logout", { method: "POST" });
```

```
    location.href = "login.html";
});
```

Dashboard.js

```
const userName = document.getElementById("userName");
const balance = document.getElementById("balance");
const accountNumber = document.getElementById("accountNumber");
const body = document.getElementById("transactionBody");

async function loadAccount() {
    const response = await fetch("/api/me");
    if (response.status === 401) {
        location.href = "login.html";
        return null;
    }
    const result = await response.json();
    userName.textContent = result.user.name;
    balance.textContent = `₹${Number(result.user.balance).toFixed(2)}`;
    accountNumber.textContent = result.user.accountNumber;
    return result.user;
}

function updateStatistics(transactions) {
    const deposits = transactions
        .filter(t => t.type === "DEPOSIT")
        .reduce((s, t) => s + Number(t.amount), 0);

    const withdrawals = transactions
        .filter(t => t.type === "WITHDRAW")
        .reduce((s, t) => s + Number(t.amount), 0);

    const transfers = transactions
        .filter(t => t.type === "TRANSFER")
        .reduce((s, t) => s + Number(t.amount), 0);

    document.getElementById("totalDeposits").textContent = `₹${deposits.toFixed(2)}`;
    document.getElementById("totalWithdrawals").textContent = `₹${withdrawals.toFixed(2)}`;
    document.getElementById("totalTransfers").textContent = `₹${transfers.toFixed(2)}`;
    document.getElementById("transactionCount").textContent = transactions.length;
}
```

```
async function loadTransactions() {
    const response = await fetch("/api/transactions");
    if (response.status === 401) {
        location.href = "login.html";
        return;
    }

    const result = await response.json();
    const transactions = result.transactions || [];
    updateStatistics(transactions);

    const recent = [...transactions].reverse().slice(0, 10);

    if (!recent.length) {
        body.innerHTML = '<tr><td colspan="4">No transactions yet.</td></tr>';
        return;
    }

    body.innerHTML = recent.map(t => `
        <tr>
          <td>${t.transactionId}</td>
          <td><span class="type ${t.type.toLowerCase()}">${t.type}</span></td>
          <td>₹${Number(t.amount).toFixed(2)}</td>
          <td>${new Date(t.date).toLocaleString()}</td>
        </tr>
    `).join("");
}

async function performAction(url, payload) {
    const response = await fetch(url, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(payload)
    });
    const result = await response.json();

    if (!result.success) {
        alert(result.message);
        return;
```

```
    }

    sessionStorage.setItem("meghanaLastTransaction", JSON.stringify(result.transaction));
    location.href = "receipt.html";
}

document.getElementById("depositBtn").addEventListener("click", () => {
    const amount = prompt("Enter deposit amount:");
    if (amount !== null) performAction("/api/deposit", { amount });
});

document.getElementById("withdrawBtn").addEventListener("click", () => {
    const amount = prompt("Enter withdrawal amount:");
    if (amount !== null) performAction("/api/withdraw", { amount });
});

document.getElementById("transferBtn").addEventListener("click", () => {
    const receiverAccount = prompt("Enter receiver Meghana account number:");
    if (!receiverAccount) return;
    const amount = prompt("Enter transfer amount:");
    if (amount !== null) performAction("/api/transfer", { amount, receiverAccount });
});

document.getElementById("refreshBtn").addEventListener("click", async () => {
    await loadAccount();
    await loadTransactions();
});

document.getElementById("logoutBtn").addEventListener("click", async () => {
    await fetch("/api/logout", { method: "POST" });
    location.href = "login.html";
});

(async function init() {
    const user = await loadAccount();
    if (user) await loadTransactions();
})();
```

Profile.js

```
const profile = document.getElementById("profile");
```

```
async function loadProfile() {
    const response = await fetch("/api/me");
    if (response.status === 401) {
        location.href = "login.html";
        return;
    }

    const result = await response.json();
    const u = result.user;

    profile.innerHTML = `
      <div class="profile-grid">
        <div><span>Full Name</span><strong>${u.name}</strong></div>
        <div><span>Username</span><strong>${u.username}</strong></div>
        <div><span>Email</span><strong>${u.email}</strong></div>
        <div><span>Mobile</span><strong>${u.mobile}</strong></div>
        <div><span>Date of Birth</span><strong>${u.dob}</strong></div>
        <div><span>Gender</span><strong>${u.gender}</strong></div>
        <div><span>Account Number</span><strong>${u.accountNumber}</strong></div>
        <div><span>Balance</span><strong>₹${Number(u.balance).toFixed(2)}</strong></div>
        <div class="wide"><span>Address</span><strong>${u.address}</strong></div>
      </div>`;
}

document.getElementById("logoutBtn").addEventListener("click", async () => {
    await fetch("/api/logout", { method: "POST" });
    location.href = "login.html";
});

loadProfile();
```

register.js

```
const form = document.getElementById("registerForm");
const message = document.getElementById("message");

form.addEventListener("submit", async (e) => {
    e.preventDefault();

    const data = Object.fromEntries(new FormData(form).entries());
```

```
    if (data.password !== data.confirmPassword) {
        message.innerHTML = '<div class="error">Passwords do not match.</div>';
        return;
    }

    delete data.confirmPassword;

    try {
        const response = await fetch("/api/register", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(data)
        });

        const result = await response.json();

        if (!result.success) {
            message.innerHTML = `<div class="error">${result.message}</div>`;
            return;
        }

        message.innerHTML =
            `<div class="success">Account created successfully!<br>
            Your account number is <strong>${result.accountNumber}</strong>.
            Redirecting to login...</div>`;

        form.reset();
        setTimeout(() => location.href = "login.html", 2500);
    } catch {
        message.innerHTML = '<div class="error">Unable to connect to server.</div>';
    }
});
```

Style.css

```
*{box-sizing:border-box;margin:0;padding:0}
body{font-family:Arial,Helvetica,sans-serif;background:#f4f9f5;color:#16241c;line-height:1.5}
a{text-decoration:none;color:inherit}
.navbar{min-height:70px;padding:0 7%;display:flex;align-items:center;justify-content:space-between;background:#0e2b1c;color:white;gap:20px}
.brand{font-size:25px;font-weight:800;letter-spacing:1px}.brand span{font-weight:400}
```

```
.nav-links{display:flex;align-items:center;gap:22px}.nav-links a:hover{opacity:.75}

.hero{min-height:540px;padding:70px 8%;display:flex;align-items:center;justify-content:space-between;gap:50px;background:linear-gradient(135deg,#0e2b1c,#1f5c3a)}

.hero-text{max-width:650px;color:white}.eyebrow{font-size:12px;letter-spacing:2px;font-weight:700;margin-bottom:10px}

.hero h1{font-size:48px;line-height:1.1;margin-bottom:20px}.hero h1 span{display:block;color:#f5d78e}

.hero-text>p:not(.eyebrow){font-size:18px;color:#dcefe0;max-width:600px}

.hero-buttons{display:flex;gap:15px;margin-top:30px}

.btn{border:0;border-radius:9px;padding:12px 20px;font-weight:700;cursor:pointer;display:inline-block}

.primary{background:#12864f;color:white}.primary:hover{background:#0d6d40}

.secondary{background:white;color:#143622}.full{width:100%;margin-top:10px}

.bank-card{width:370px;height:220px;border-radius:20px;padding:25px;background:linear-gradient(135deg,#1c4a2e,#0e2b1c);color:white;box-shadow:0 25px 60px #0006;flex-shrink:0}

.card-top,.card-bottom{display:flex;justify-content:space-between;font-weight:700}.chip{width:45px;height:34px;border-radius:7px;background:#e0c169;margin-top:35px}

.card-number{font-size:20px;letter-spacing:3px;margin:20px 0}

.features{display:grid;grid-template-columns:repeat(3,1fr);gap:22px;padding:50px 8%;background:white}

.feature{padding:25px;border-radius:15px;background:#f5f8fc}.feature h3{margin-bottom:8px;color:#15703f}

footer{text-align:center;padding:25px;background:#0e2b1c;color:#cfe6d4}

.form-page{min-height:100vh}.form-container,.login-box,.profile-container,.receipt{width:min(850px,92%);margin:45px auto;background:white;padding:35px;border-radius:18px;box-shadow:0 10px 35px #10253a14}

.form-container h1,.login-box h1,.profile-container h1,.receipt h1{text-align:center;margin-bottom:5px}

.muted{text-align:center;color:#718096;margin-bottom:25px}.grid-2{display:grid;grid-template-columns:1fr 1fr;gap:18px}

label{display:block;font-weight:700;margin-bottom:15px}input,select,textarea{width:100%;margin-top:7px;padding:12px;border:1px solid #ccd6e2;border-radius:8px;font:inherit;background:white}

textarea{resize:vertical}.center{text-align:center;margin-top:20px}.center a{color:#15703f;font-weight:700}

.success,.error{padding:12px;border-radius:8px;margin:12px 0}.success{background:#e7f8ed;color:#176b37}.error{background:#fdebec;color:#a32832}

.login-box{max-width:460px}.login-icon{text-align:center;font-size:35px}.security-note{text-align:center;margin-top:18px;color:#65758b;font-size:13px}

.logout-btn{border:0;background:#b52d3b;color:white;padding:9px 14px;border-radius:7px;cursor:pointer;font-weight:700}

.dashboard{width:min(1200px,92%);margin:35px auto}.dashboard-heading{display:flex;justify-content:space-between;align-items:center;margin-bottom:25px}

.dashboard-heading h1{font-size:30px}.secure-badge{background:#e5f7ec;color:#17713c;padding:8px 13px;border-radius:20px;font-size:13px;font-weight:700}

.account-card{padding:30px;border-radius:20px;background:linear-gradient(135deg,#186b3d,#0e2b1c);color:white;display:flex;justify-content:space-between;align-items:center}

.account-card p{font-size:12px;letter-spacing:1px;opacity:.8}.account-card h2{font-size:38px;margin:5px 0}.account-mark{font-size:35px;font-weight:800}

.stats-grid{display:grid;grid-template-columns:repeat(4,1fr);gap:15px;margin:20px 0}.stat-card{background:white;padding:20px;border-radius:13px;box-shadow:0 5px 20px #10253a0c}.stat-card span{display:block;color:#718096;font-size:13px}.stat-card strong{font-size:21px;display:block;margin-top:6px}

.actions{display:grid;grid-template-columns:repeat(3,1fr);gap:18px;margin:20px 0}.action-card{background:white;padding:25px;border-radius:15px;box-shadow:0 5px 20px #10253a0c}.action-card p{color:#718096;margin:7px 0 18px}

.transaction-section{background:white;border-radius:15px;padding:25px}.section-head{display:flex;justify-content:space-between;align-items:center;margin-bottom:18px}

.table-wrap{overflow-x:auto}table{width:100%;border-collapse:collapse}th,td{padding:13px;text-align:left;border-bottom:1px solid #e8edf3;font-size:14px}th{background:#f7f9fc}

.type{font-weight:700;font-size:12px}.deposit,.received{color:#138345}.withdraw,.transfer{color:#b15a00}

.profile-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px;margin-top:25px}.profile-grid>div{padding:18px;background:#f7f9fc;border-radius:10px}.profile-grid span,.receipt-row span{display:block;color:#718096;font-size:12px}.profile-grid strong{display:block;margin-top:4px}.wide{grid-column:1/-1}
```

```
.receipt{max-width:600px;text-align:center}.receipt-logo{font-size:27px;font-weight:800;color:#15703f;margin-bottom:20px}.receipt-
status{color:#17713c;font-weight:800;margin-bottom:15px}.receipt-row{display:flex;justify-content:space-between;gap:20px;text-align:left;padding:14px
0;border-bottom:1px solid #e7edf4}.receipt-buttons{display:flex;justify-content:center;gap:12px;margin-top:25px}

@media(max-width:800px){

.navbar{padding:15px 5%;flex-wrap:wrap}.nav-links{gap:12px;flex-wrap:wrap}

.hero{padding:45px 6%;flex-direction:column;align-items:flex-start}.hero h1{font-size:38px}.bank-card{width:100%;max-width:370px}

.features,.actions,.stats-grid{grid-template-columns:1fr}.grid-2,.profile-grid{grid-template-columns:1fr}.wide{grid-column:auto}

.dashboard-heading{align-items:flex-start;gap:15px;flex-direction:column}

.account-card h2{font-size:30px}

.form-container,.login-box,.profile-container,.receipt{padding:25px}

}

@media(max-width:480px){

.nav-links{font-size:13px}.hero-buttons{flex-direction:column;width:100%}.hero-buttons .btn{width:100%;text-align:center}

.bank-card{height:auto;min-height:200px}.card-number{font-size:15px}

.dashboard{width:94%}.transaction-section{padding:15px}

th,td{font-size:12px;padding:9px}

.receipt-row{font-size:13px}

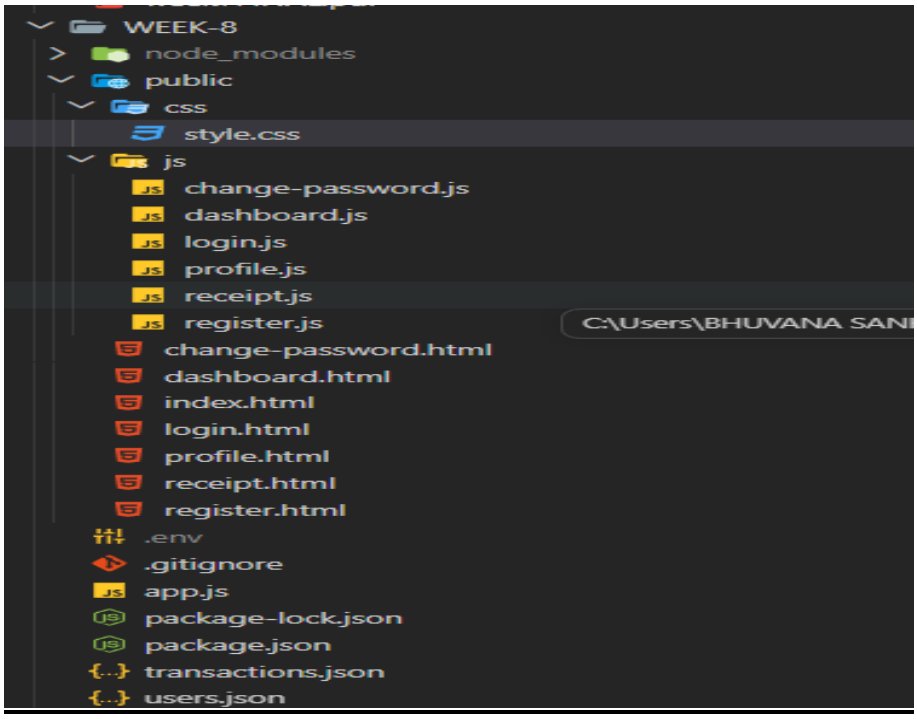
}

@media print{

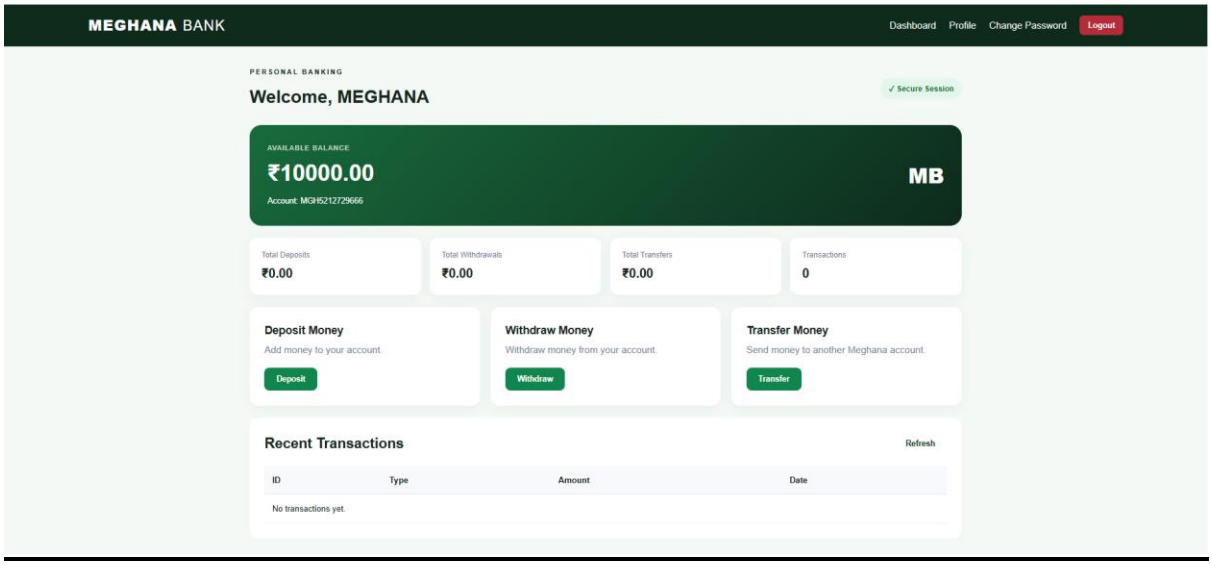
.receipt-buttons{display:none}.receipt{box-shadow:none;margin:0;width:100%}body{background:white}

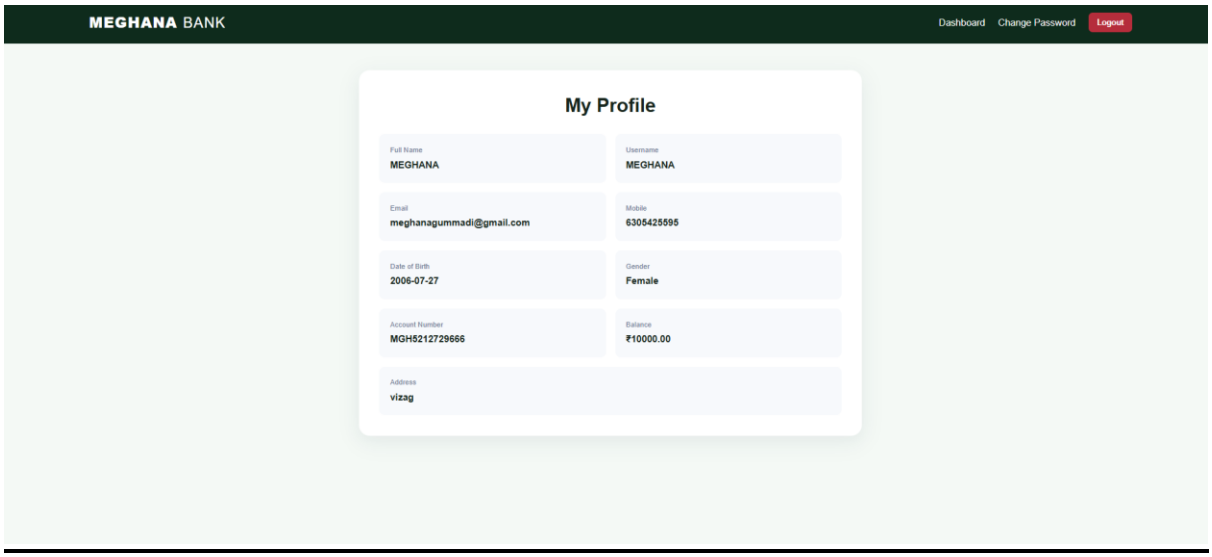
}
```

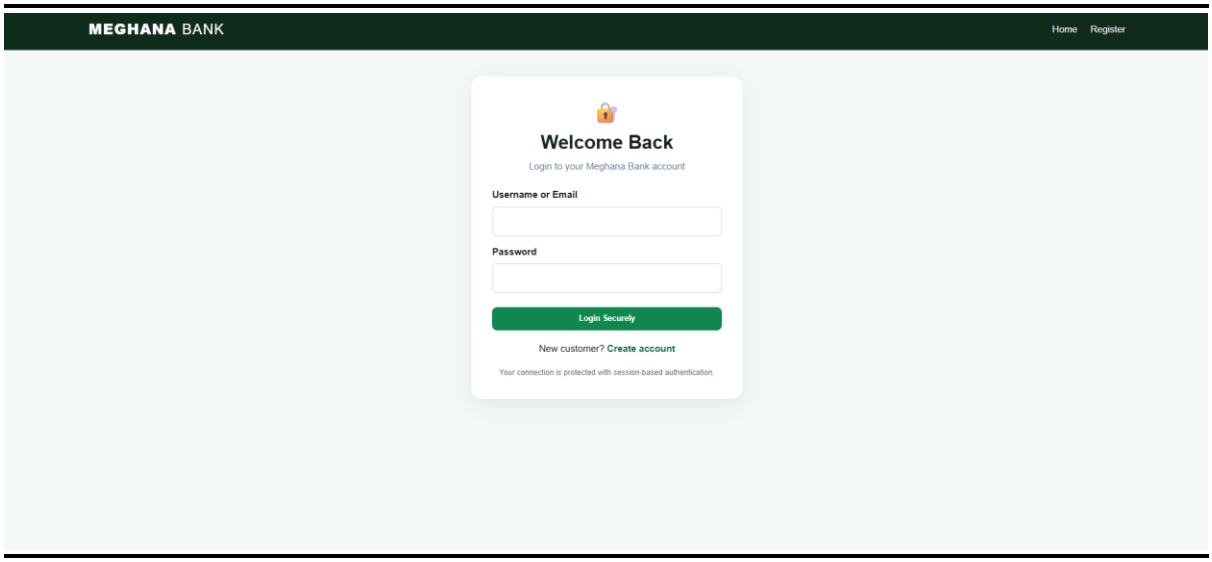
FOLDER STRUCTURE

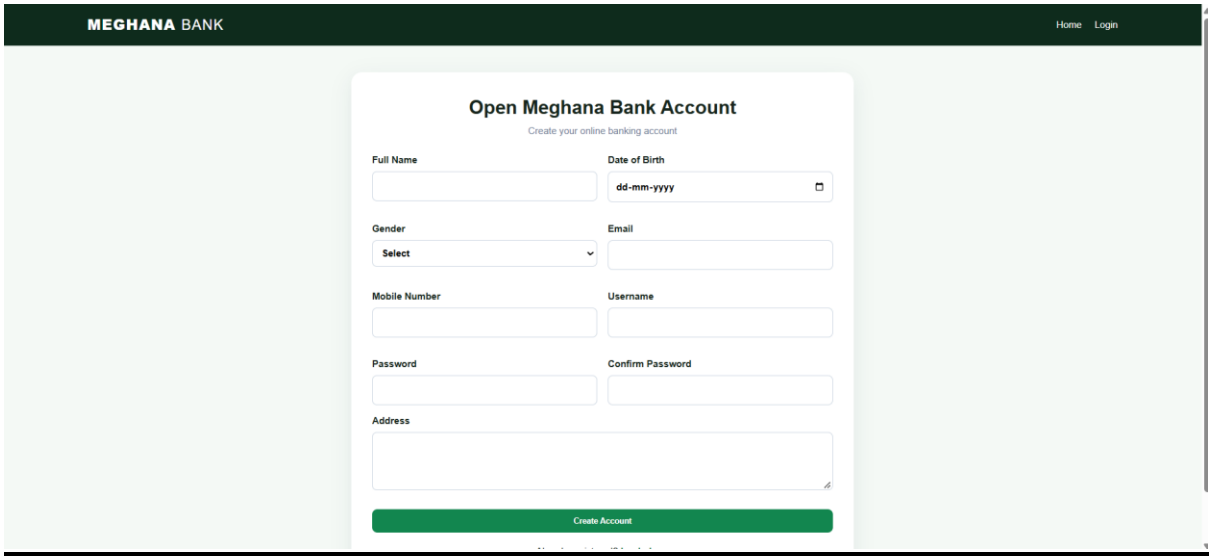


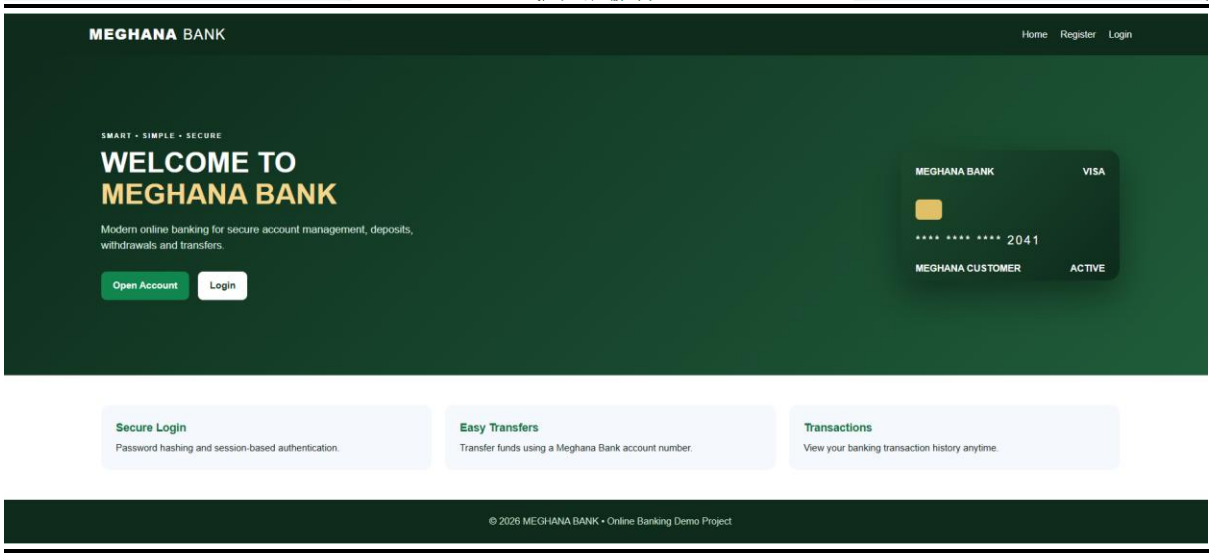
OUTPUTS:


MEGHANA BANK
Dashboard
Profile
Change Password
Logout
PERSONAL BANKING
Welcome, MEGHANA
✓ Secure Session
AVAILABLE BALANCE
₹10000.00
Account: MGH5212729666
MB
Total Deposits
₹0.00
Total Withdrawals
₹0.00
Total Transfers
₹0.00
Transactions
0
Deposit Money
Add money to your account.
Deposit
Withdraw Money
Withdraw money from your account.
Withdraw
Transfer Money
Send money to another Meghana account.
Transfer
Recent Transactions
Refresh
ID
Type
Amount
Date
No transactions yet.


MEGHANA BANK
Dashboard
Change Password
Logout
My Profile
Full Name
MEGHANA
Username
MEGHANA
Email
meghanagummadi@gmail.com
Mobile
6305425595
Date of Birth
2006-07-27
Gender
Female
Account Number
MGH5212729666
Balance
₹10000.00
Address
vizag


MEGHANA BANK
Home
Register
Welcome Back
Login to your Meghana Bank account
Username or Email
Password
Login Securely
New customer? Create account
Your connection is protected with session-based authentication.


MEGHANA BANK
Home
Login
Open Meghana Bank Account
Create your online banking account
Full Name
Date of Birth
dd-mm-yyyy
Gender
Select
Email
Mobile Number
Username
Password
Confirm Password
Address
Create Account


MEGHANA BANK
Home
Register
Login
SMART • SIMPLE • SECURE
WELCOME TO
MEGHANA BANK
Modern online banking for secure account management, deposits, withdrawals and transfers.
Open Account
Login
MEGHANA BANK
VISA
**** **** **** 2041
MEGHANA CUSTOMER
ACTIVE

Secure Login
Password hashing and session-based authentication.
Easy Transfers
Transfer funds using a Meghana Bank account number.
Transactions
View your banking transaction history anytime.

© 2026 MEGHANA BANK • Online Banking Demo Project